Client-Side ANN on the Web: A Wikipedia Search Demonstration

David Denisov
University of Haifa
Haifa, Israel

Live search demo: https://wiki-ann-demo-20261022.pages.dev/client_wiki/
Clustering lab: https://wiki-ann-demo-20261022.pages.dev/client_wiki/benchmark.html
Code and benchmarks: https://github.com/DavidDenisov/ANN-on-the-Web

Abstract

We demonstrate browser-based approximate nearest neighbour (ANN) search over 35.2 million English Wikipedia passages served as static Web files. The browser embeds questions locally, uses inverted file (IVF) routing to select vector clusters for parallel download, ranks their vectors, and fetches text for the top results. This avoids a full index download or an application search server. Visitors can submit questions and inspect passages, timings, and download costs; a companion lab exposes the recovery–bandwidth trade-off. At 32k clusters, SQ6 centroids, and four probes, a fixed question returning ten passages averages 130.3 ms, including embedding and text loading, with 3.25 MB of compressed payload. Worker startup averages 2.46 s. At the same settings, synthetic perturbations of L2 magnitude 0.4 yield 98.1% source recovery@1; this is not real-question recall. Queries and embeddings stay local, while file requests can reveal access patterns.

CCS Concepts

• **Information systems** → **Similarity search**.

Keywords

approximate nearest neighbors, semantic search, client-side retrieval

ACM Reference Format:
David Denisov. 2027. Client-Side ANN on the Web: A Wikipedia Search Demonstration. In *Companion Proceedings of the ACM Web Conference 2027 (WWW '27).* ACM, New York, NY, USA, 4 pages.

1 Introduction

Semantic search on a static document collection usually requires a search service or a large local index. We demonstrate browser ANN over 35.2 million passages without either: local inverted file (IVF) routing selects remote cluster files for parallel download. The probe count controls the number of clusters fetched and ranked.

Our contribution is a static-Web deployment and an interactive view of its costs, using established ANN techniques. The full FP16 vectors occupy about 27 GB; only selected clusters and result text cross the network per query. Cloudflare Pages hosts the client and sharded index.

1.1 Related work

SPANN [1] keeps centroids in memory and posting lists on disk; our browser applies this established partitioned-search pattern to static HTTP files. Tiptoe [4] also selects semantic clusters locally but uses cryptography to hide the selected cluster from its servers. Our requests reveal access patterns. Static selective loading also appears in Pagefind's lexical search and sql.js-httpvfs's range-based database reads.

MeMemo [11] and WebANNS [6] use browser-local HNSW in RAM and IndexedDB; DiskANN [9] uses graph traversal. WebANNS evaluates up to 480,000 passages. Our larger remote corpus uses parallel cluster reads; the ANN algorithm is established.

2 Demonstration scenario

The two public pages use the same Wikipedia passage collection.

Wikipedia search. Visitors enter a question or other text and receive ranked passages with links to Wikipedia articles. Defaults are 32k clusters, SQ6 centroids, four probes, and ten results. Controls offer three cluster counts, six centroid formats, and 1–100 probes. The page reports embedding, centroid loading, retrieval, and text-loading times alongside vector downloads. The walkthrough compares one and four probes for the same question, inspects changed passages, and then invites audience questions. Figure 1 shows the actual interface. The artifact adds a bandwidth notice for large probe budgets; queries can be cancelled.

Clustering lab. Visitors vary clustering, probes, centroid format, and query noise to compare recovery, downloads, and speed. The current lab (Figure 2) fixes each configuration to 10,000 queries, resolving recovery in 0.01 percentage-point steps. Visitors compare one and four probes at 16k, then centroid formats, after inspecting noise and download costs. Precomputed vectors exclude embedding from timings.

Public access. Both pages require Internet access; search also requires WebAssembly. A poster QR code links to the demo; a laptop walkthrough and saved screenshots support the presentation. The source repository includes the client, builders, figures, and compact measurements. Live hosting depends on the CDN; the artifact documents self-hosting and does not promise permanent preview URLs.

WWW '27, Dublin, Ireland
2027.

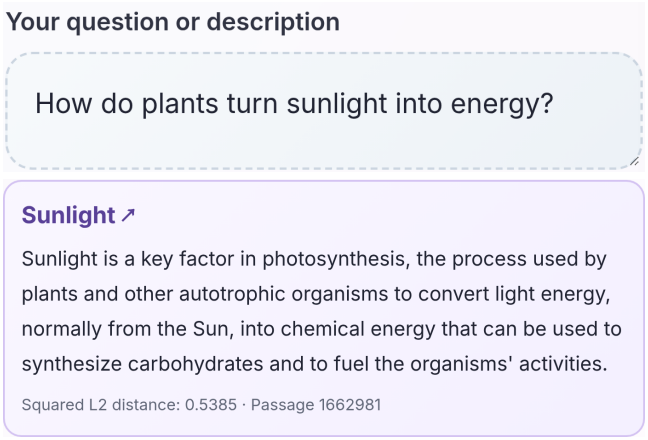


Figure 1: Live search.

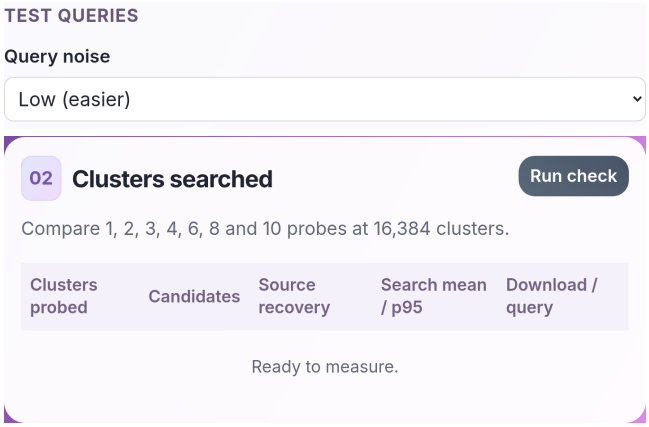


Figure 2: Live lab, before a run.

3 Method outline

Figure 3 traces local computation and the static files downloaded at startup and during search.

The Maloyan dataset [7] provides the passages and their 384-dimensional `all-MiniLM-L6-v2` embeddings [8, 10].

Transformers.js [3] loads the `Xenova/all-MiniLM-L6-v2` model in q8 format. Queries are truncated to 256 tokens.

Index preparation

We train C centroids with Faiss k-means [2], using 20 iterations on about 2.1 million uniformly sampled passages. We assign each passage to its nearest centroid and store its ID and FP16 vector in that centroid's gzipped file. Centroids are published in FP16 and compressed formats; SQ6 uses six-bit scalar quantisation per component. Passage vectors remain FP16, occupying about 27 GB before compression.

Result text. Brotli files hold 500 passages each, except the last. The browser fetches result batches and keeps decoded text in an 8 MiB cache.

Static hosting. The 70,336 text files use 71 branch deployments; vectors and centroids use 59 more in a second Pages project. Manifests map files to URLs. The uploader chooses batches of 1,000 files, below the provider's 20,000-file Free-plan ceiling; 1,000 is our packaging choice, not that ceiling. Object storage can replace branch hosts by updating manifests and preserving CORS, content-encoding, and worker-isolation headers.

4 Supporting evaluation

All three indexes cover 35,167,920 passage vectors. We compare routing settings using synthetic embedding perturbations and measure the complete ten-result worker workflow with a fixed question.

Query setup. All configurations share 10,000 queries from uniformly sampled distinct source passages. We add $v \in \mathbb{R}^{384}$ with uniformly random direction and $\|v\|_2 = \alpha = 0.4$ to each unit-length source vector, then renormalise. The paper reports the largest noise preset; the lab also offers 0.05 and 0.15.

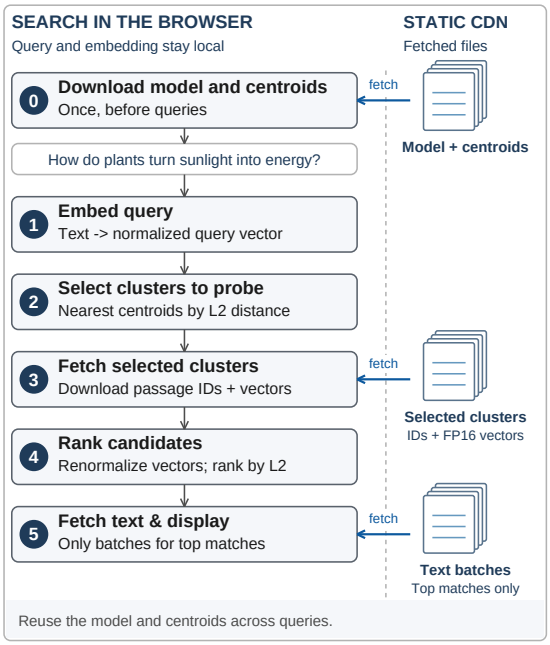


Figure 3: Local query embedding and ranking, with model and centroid downloads at startup and selective vector and text downloads per query.

Perturbation scale and scope. On 1,000 sources, mean query-source L2 displacement is 0.378, or 55.5% of the mean nearest-neighbour distance (0.681) from an exact full-corpus scan excluding self matches. This controlled test exercises IVF boundary routing; isotropic passage noise does not model the question–passage distribution gap.

Metric. **Source recovery@1** is the fraction of perturbed queries whose source passage ranks first. Noise can move another passage ahead of the source even in exact search, so the plateau is not an ANN recall ceiling. Real-question recall@10 against exact search, relevance, and q8 query-encoder effects remain unevaluated. For SQ6 at four probes, the 95% Wilson interval [12] is [97.77, 98.31]% (9,806/10,000); intervals describe sampling uncertainty under this synthetic protocol.

Test environment. Experiments use Ubuntu 22.04 on an Intel Core i5-12400F PC with 32 GB RAM and a GTX 1660 SUPER GPU. GPU Faiss prepared the indexes. Browser experiments use Chromium 155; inference uses Transformers.js 4.3.1 with CPU-only WebAssembly and four threads. Separate FAST.com checks reported about 300 Mbit/s download; CDN throughput was uncontrolled.

Embedding compatibility. Re-embedding 1,000 uniformly sampled passages with the q8 browser encoder yields mean and 5th-percentile cosine similarities of 0.9924 and 0.9892 to the stored vectors; this passage check does not validate question retrieval.

We compare $C \in \{8{,}192, 16{,}384, 32{,}768\}$ with 1, 4, and 8 probes. At 32k, we sweep 1, 2, 3, 4, 6, 8, and 10 probes, and compare six centroid formats at four probes: FP16, SQ8/SQ6/SQ4 and symmetric INT8 [2], and PQ4 with 4-bit codes per eight-component block [5]. Cluster assignments and FP16 passage vectors stay fixed. Shared configurations run once.

Hosting and timing. Vector and text files are fetched before timing to prime the CDN cache; the fixed-question run uses 100 preliminary searches. CDN residency and connections remain uncontrolled; cache bypass does not establish a cold CDN. Timed searches fetch up to ten clusters in parallel. Timed fetches bypass browser HTTP caching, and text loads also bypass the decoded-text cache; untimed accuracy checks allow cached cluster files.

Retrieval time covers routing, cluster download, decompression, decoding, and ranking; Section 4.3 reports startup and complete queries including embedding and text loading. Each query configuration loads its centroids once. Centroid download and decoding use 1,000 trials per file. Tables report mean and 95th-percentile times (p95), and vector downloads in decimal MB (10^6 gzipped payload bytes). Configuration order was randomised; the original runner records its seed, and the 32k supplement records its shuffled order.

4.1 Cluster count and probe budget

With FP16 centroids, four probes recover over 98% of source passages at all three cluster counts (Table 1). More clusters reduce vector downloads and retrieval time but increase the initial centroid load.

At 32k clusters, increasing from four to eight probes raises source recovery from 98.1% to 98.8%, while increasing mean retrieval time 1.78× and vector downloads 1.96× (Figure 4, Table 1).

4.2 Centroid loading costs

Table 2 compares six centroid formats at 32k clusters and four probes. **SQ6 reduces mean centroid loading time from** 929 **ms (FP16) to** 324 **ms**, with 98.1% recovery for both.

Table 1: FP16 four-probe retrieval at large noise: source recovery@1, mean / p95 time, and vector payload per query. 10,000 **queries per setting.**

Clusters	Probes	Recovery (%)	Time (ms)	MB/query
8192	4	98.4	462 / 662	13.65
16384	4	98.2	215 / 326	7.11
32768	4	98.1	135 / 182	3.98

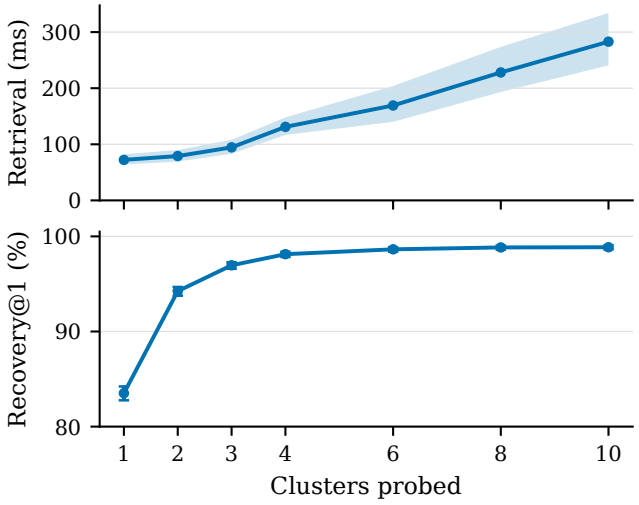


Figure 4: FP16 probe sweep at 32k clusters, large noise. Top: median retrieval time and interquartile band. Bottom: source recovery@1 with 95% **Wilson intervals.** 10,000 **queries per setting.**

Table 2: Centroid formats at 32k clusters and four probes: source recovery@1 on 10,000 large-noise queries, file size, and loading mean / p95 over 1,000 trials per file. Decode includes gzip decompression.

Format	Recovery (%)	Size (MB)	Fetch (ms)	Decode (ms)
FP16	98.1	23.24	675 / 692	254 / 260
SQ8	98.2	11.23	300 / 339	113 / 115
SQ6	98.1	8.98	224 / 261	100 / 102
SQ4	97.4	4.86	83 / 102	88 / 91
INT8	98.1	10.99	290 / 342	103 / 106
PQ4	68.0	0.81	23 / 33	30 / 31

FP16, SQ8, SQ6, and INT8 differ by at most 0.09 percentage points; these marginal intervals do not establish a quality ranking. SQ4 and PQ4 reduce recovery. The SQ4 fetch implies 468 Mbit/s, above the separate 300 Mbit/s speed check: network throughput was not capped, so fetch times are observations rather than controlled bandwidth comparisons.

These results motivate 32k/SQ6 and four probes: reduced vector downloads and centroid loading costs under this synthetic protocol.

4.3 Client setup and query costs

Complete ten-result query. For "How do plants turn sunlight into energy?", the client returns ten passages at 32k/SQ6 and four probes. Across 1,000 repetitions, search submission to worker reply takes 130.3/150.3 ms (mean/p95), including embedding, vector retrieval, text loading, and message handling. Startup and DOM rendering are excluded; model and decoded centroids stay loaded. This repeated fixed-question p95 measures jitter, not variation across questions. The results require 2.48 MB of gzipped vectors and 0.77 MB of Brotli text (3.25 MB total payload, excluding HTTP headers).

Separate components. For synthetic queries, top-1 retrieval at 32k/SQ6 and four probes takes 122/187 ms (mean/p95), with 98.1% source recovery and 3.98 MB of vectors. The fixed question transfers about 38% fewer vector bytes; these component runs cannot be added to estimate varied-question end-to-end latency.

Startup. Startup spans worker creation until the model and 32k SQ6 centroids are ready, excluding page navigation. Across 100 fresh browser contexts (reusing the Chromium process), it takes 2.46/2.53 s (mean/p95). The model (23.7 MB), WebAssembly runtime (14.3 MB), and centroids (9.0 MB) total about 47 MB. Browser caching is bypassed; loading phases overlap.

4.4 Trade-offs and limitations

A hypothetical 48-byte/vector PQ index needs 1.69 GB for codes alone. Equating this with $47 + 3.25Q$ MB gives about 505 queries, or 413 using 3.98 MB of vectors/query before text. This illustrates byte budgets, not equal-quality or latency baselines: IDs, codebooks, caching, and storage overhead change the crossing. One million uncached fixed-question searches transfer 3.25 TB before startup or headers. A server returns text alone but receives the query and needs search compute; that baseline is unmeasured.

FP16 passage vectors dominate transfer. SQ8/PQ passage codes with optional rescoring could reduce it, but change ranking and require evaluation. At 32k, manifest cluster sizes have median 1,023, p95 2,188, and maximum 8,263 passages. The four largest gzip files sum to 19.32 MB, an upper bound at four probes, not an observed query percentile; byte tails remain unmeasured. Varied real-question end-to-end mean/p95/max, cold-connection and cold-CDN trials, and throttled/mobile profiles remain necessary before generalising the desktop timings.

Privacy threat model. Assuming the served client code is trusted, query text and embeddings stay in the browser. The host/CDN can associate IP addresses and timing with four cluster IDs among 32,768, revealing semantic regions, and result-batch IDs, each identifying a set of at most 500 passages. These requests are not cryptographically private and can reveal likely results, though not which displayed result was read. Larger batches or decoy requests add bandwidth without guaranteeing privacy; PIR, as in Tiptoe [4], requires a different serving protocol. We do not measure inference success from these observations.

5 Conclusion

Local IVF routing enables browser search over 35.2 million passages through selective downloads from static hosting. The search page and lab expose how clustering, probing, and centroid compression affect source recovery, latency, and bandwidth.

6 Ethical use of data

Wikipedia passages remain subject to their CC BY-SA attribution and share-alike terms; article links credit their contributors. Maloyan supplies the embeddings [7], but its dataset card specifies no separate licence; that redistribution status needs clarification. Passages reflect an archived snapshot and may be outdated or biased; linked articles may have changed or been deleted. Experiments use sampled passages and synthetic embedding perturbations, with no human-participant study.

References

[1] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: Highly-efficient Billion-scale Approximate Nearest Neighborhood Search. In *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan (Eds.), Vol. 34. Curran Associates, Inc., 5199–5212. https://proceedings.neurips.cc/paper_files/paper/2021/file/299dc35e747eb77177d9cea10a802da2-Paper.pdf

[2] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2026. The Faiss Library. *IEEE Transactions on Big Data* 12, 2 (2026), 346–361. doi:10.1109/TBDATA.2025.3618474

[3] Hugging Face. 2024. Transformers.js: State-of-the-art Machine Learning for the Web. GitHub repository. https://github.com/huggingface/transformers.js Accessed October 24, 2026.

[4] Alexandra Henzinger, Emma Dauterman, Henry Corrigan-Gibbs, and Nickolai Zeldovich. 2023. Private Web Search with Tiptoe. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. ACM, 396–416. doi:10.1145/3600006.3613134

[5] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128. doi:10.1109/TPAMI.2010.57

[6] Mugeng Liu, Siqi Zhong, Qi Yang, Yudong Han, Xuanzhe Liu, and Yun Ma. 2025. WebANNS: Fast and Efficient Approximate Nearest Neighbor Search in Web Browsers. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '25)*. Association for Computing Machinery, New York, NY, USA, 2483–2492. doi:10.1145/3726302.3730115

[7] Narek Maloyan. 2023. Wikipedia-22-12-en embeddings with all-MiniLM-L6-v2. Hugging Face dataset. https://huggingface.co/datasets/maloyan/wikipedia-22-12-en-embeddings-all-MiniLM-L6-v2 Accessed October 24, 2026.

[8] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Association for Computational Linguistics, Hong Kong, China, 3982–3992. doi:10.18653/v1/D19-1410

[9] Suhas Jayaram Subramanya, Devvrit, Rohan Kadekodi, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2019. DiskANN: fast accurate billion-point nearest neighbor search on a single node. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[10] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. In *Advances in Neural Information Processing Systems*, Vol. 33. Curran Associates, Inc., Red Hook, NY, USA, Article 485, 13 pages.

[11] Zijie J. Wang and Duen Horng Chau. 2024. MeMemo: On-device Retrieval Augmentation for Private and Personalized Text Generation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '24)*. Association for Computing Machinery, New York, NY, USA, 2765–2770. doi:10.1145/3626772.3657662

[12] Edwin B. Wilson. 1927. Probable Inference, the Law of Succession, and Statistical Inference. *J. Amer. Statist. Assoc.* 22, 158 (June 1927), 209–212. doi:10.1080/01621459.1927.10502953